

Especificação do Trabalho II – ELC1018 Sistemas Distribuídos

Programação de um middleware para Comunicação *multicast* com Ordenamento Causal de mensagens e Mecanismo de Estabilização para descarte de mensagens do buffer baseado em vetor de relógios lógicos

O trabalho, que deve ser feito em dupla, consiste em **implementar um *middleware* para envio de mensagens *multicast* obedecendo à ordem causal e realizando o controle de estabilização de mensagens para descarte das mensagens do buffer**. Consulte o material teórico (no final deste documento) e os slides (no Moodle) para compreender os **algoritmos**.

O *middleware* deve ser um único *packet* Java nomeado `CausalMulticast` que será importado por algum programa Java (usuário) e que oferecerá através de sua API a facilidade de enviar uma mensagem *multicast* para um conjunto de destinatários (grupo) obedecendo o ordenamento causal. Vide detalhes da API abaixo. O *middleware* deve implementar a comunicação *multicast* através do envio de mensagens *unicast* não confiável (sockets UDP) e implementar um Serviço de Descoberta que deve usar IP *multicast* para realizar a descoberta das instâncias de usuário que executam o *middleware* (participantes da computação/grupo). Neste trabalho você não deve usar RMI.

A API do *middleware* deve oferecer um método `mcsend(msg, this)`, para os usuários enviarem mensagens *multicast* com ordenamento causal, e um método `deliver(msg)`, para o usuário receber mensagens. Note que o parâmetro `this` do método `mcsend` é quem passa a referência do objeto remoto do usuário para que a resposta seja recebida por *callback* via método `deliver`. O Serviço de Descoberta deve permanecer sempre ativo em cada instância do *middleware*, a fim de permitir atualização dinâmica dos membros do grupo.

Importação do pacote `CausalMulticast`:

```
import CausalMulticast.*;
```

Interface que deve ser implementada por todo usuário do pacote `CausalMulticast`:

```
Public interface ICausalMulticast {  
    public void deliver(String msg);  
}
```

API oferecida pelo pacote `CausalMulticast`:

```
public CausalMulticast(String ip, Integer port, ICausalMulticast client)  
public void mcsend(String msg, ICausalMulticast cliente)
```

Para possibilitar a correção do trabalho, faça o envio de cada mensagem *unicast* ser controlado via teclado, ou seja, deve haver uma pergunta antes de cada envio *unicast* (controle) questionando se é para enviar a todos ou não. O caso “não” deve permitir atrasar uma única mensagem escolhida pelo professor/aluno e deve permitir o envio posterior da mensagem. Por exemplo, com três processos, deve-se poder escolher para qual deles vamos atrasar a mensagem. O conteúdo do buffer (mensagens) e dos relógios lógicos também precisam ser permanentemente demonstrados na tela/terminal. Não é necessário implementar uma GUI na aplicação do usuário.

Os algoritmos de ordenação causal e estabilização de mensagens a serem implementados podem ser os propostos no artigo *Fundamentals of Distributed Computing: A Practical Tour of Vector Clock Systems*, disponível em Slides no Moodle e na [Web](#). Porém outros algoritmos com vetores de relógios lógicos também podem ser implementados (indique isto na documentação, se ocorrer).

Os algoritmos básicos estão descritos na próxima página.

Na avaliação, o seu *middleware* será testado com um cliente do professor! Respeite as especificações!

A entrega do trabalho deve ser no Moodle e deve incluir código e documentação no formato Oxygen ou JavaDoc.

```

procedure mcsend(msg)
    msg.VC  $\leftarrow$  VCi                                     % constrói o timestamp da msg
     $\forall x \in \{1, \dots, n\}$  do send(msg) to Px enddo      % multicast msg
    VCi[i]  $\leftarrow$  VCi[i]+1                               % Pi fez mais um broadcast

when Pi receives msg from Pj                           % relógio msg.VC por piggyback
    % verifica e realiza o ordenamento causal
    atrasa entrega até ( $x \in \{1, \dots, n\} : \text{msg.VC}[x] \leq \text{VC}_i[x]$ )
    % atualiza variável de controle e entrega msg
    if i  $\neq$  j then VCi[j]  $\leftarrow$  VCi[j]+1
    entrega msg para a aplicação

```

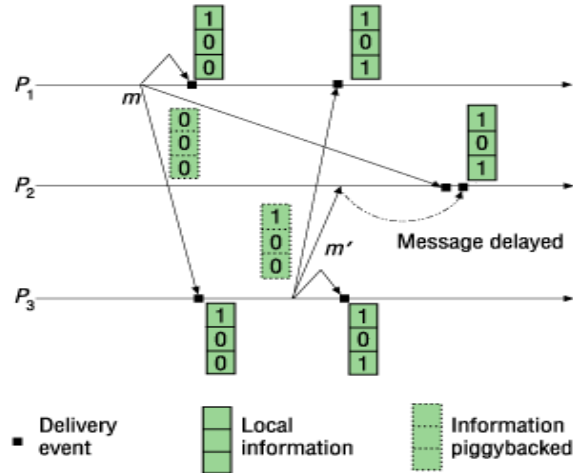


Figura 1 – Funcionamento do algoritmo para ordenamento causal.

```

procedure mcsend(msg)                                     % realizado por Pi
    msg.VC  $\leftarrow$  MCi[i][*]                               % constrói o timestamp de msg
    msg.sender  $\leftarrow$  i
    for all P do send(msg) to Pj enddo                  % multicast
    MCi[i][i]  $\leftarrow$  MCi[i][i]+1                         % Pi fez mais um multicast

when Pi receives msg from Pj                           % msg traz msg.VC por piggyback
    deposit(msg)                                           % coloca msg no buffer
    MCi[j][*]  $\leftarrow$  msg.VC                               % atualiza visão do Pi com visão de Pj
    if i  $\neq$  j then MCi[i][j]  $\leftarrow$  MCi[i][j]+1          % mais 1 msg de Pj entregue
    deliver msg to the upper layer                       % evento de entrega

when (existe msg no bufferi AND msg.VC[msg.sender]  $\leq$  min1 ≤ x ≤ n(MCi[x][msg.sender]))
    discard(msg)                                           % elimina msg do buffer local

```

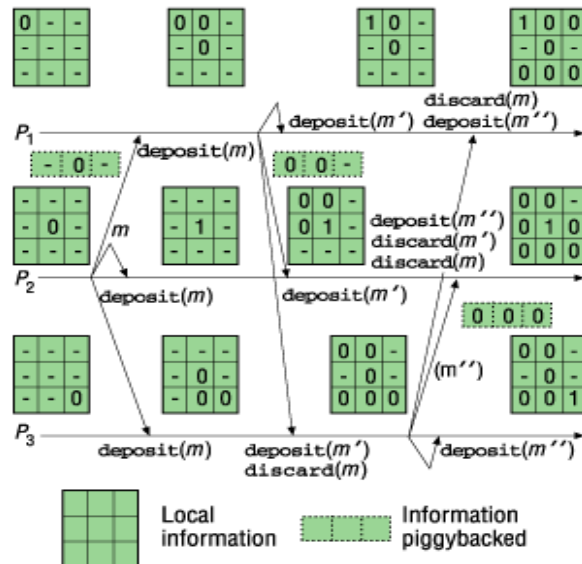


Figura 2 – Funcionamento do algoritmo para estabilização de mensagens.